

Skills Nobody Taught

Transport-Metric Skill Discovery on Granular Media: the complete self-study report

Azfar Azdi Arfakhsyad

July 2026

Abstract

This report documents, from first principles, a research campaign into *unsupervised skill discovery* — teaching a robot to invent useful behaviors with no rewards and no demonstrations — applied for the first time to *granular material* (an excavator digging soil). It is written to be self-contained: every mathematical tool (probability, entropy, mutual information, reinforcement learning, optimal transport, constrained optimization) is built up from zero before it is used. We cover the three landmark prior methods (DIAYN, DADS, METRA) and their flaws; our proposed method (TMSD) and its two ingredients; all nine experiments with their data; the falsification of our own original hypothesis; the discovery that replaced it (a two-ingredient law and a “slow-state” failure taxonomy); and an honest account of what the method can and cannot do. 33 training runs, ≈ 70 GPU-hours, one desktop GPU.

How to read this report. Sections 2–3 build the mathematical toolkit (skip them if you know information theory and RL; return whenever a symbol bites). Section 4 is the intellectual history — three methods, three flaws. Section 5 builds optimal transport from a sand pile. Section 6 states our method. Section 7 is the heart: nine experiments in chronological order, including the death of our own hypothesis and what replaced it. Sections 8–9: what is new, what is broken, what is next. Boxes: **blue** = plain-English intuition, **gray** = worked numeric example, **green** = a result to remember, **red** = a flaw that matters later. Every figure is generated from the actual experiment data (`report/make_figures.py`).

Contents

1	The problem in plain language	3
2	Mathematical toolkit I: probability, surprise, and information	3
2.1	Random variables, distributions, expectation	3
2.2	Entropy: measuring uncertainty	3
2.3	Conditional entropy and mutual information	4
3	Mathematical toolkit II: reinforcement learning in one section	5
3.1	The loop	5
3.2	Value functions and the critic	6
3.3	SAC, the workhorse we use everywhere	6
3.4	Skill-conditioning	6
4	The three landmark methods, and the flaw in each	6
4.1	DIAYN (2019): reward being identifiable	6
4.2	DADS (2020): reward being predictable	8
4.3	METRA (2024): reward going far	9

5	Mathematical toolkit III: optimal transport	10
5.1	Three rulers for the same question	11
5.2	The earth mover’s problem	12
5.3	The formal definition	12
5.4	The one-dimensional miracle	13
6	Our method: TMSD	14
6.1	The two ingredients	14
6.2	What we believed, and what we pre-registered	14
6.3	The testbed	15
6.4	How the state is represented: four layers	15
7	The experiments	15
7.1	Experiment 1 — Does anything emerge at all?	15
7.2	Experiment 2 — Is the transport metric doing the work?	16
7.3	Experiment 3 — Then what IS doing the work?	17
7.4	Experiment 4 — The metric’s second chance (2D measure)	17
7.5	Experiment 5 — Composing skills into goals (zero-shot shaping)	18
7.6	Experiment 6 — DIAYN on soil (the MI 2×2)	19
7.7	Experiment 7 — DADS on soil	21
7.8	The unifying law: materials are slow states	21
7.9	Experiment 8 — Where it honestly breaks (and why)	22
7.10	The full frontier comparison	22
8	What is actually new here (novelty statement)	24
9	Limitations and what comes next	26
A	Appendix: reproduction guide	27

1 The problem in plain language

Robots today learn tasks the way a dog learns tricks: a human defines a *reward* — a number that goes up when the robot does the right thing — and the robot fumbles around until it finds behavior that makes the number go up. This works, but it has a painful catch: **someone has to write the reward**, and for every new task, a new reward. Writing rewards is genuinely hard. What is the reward for “knead this dough into a ball”? For “grade this slope nicely”? Engineers spend more time debugging reward functions than any other part of a robot-learning system, because a robot will ruthlessly exploit any loophole in a sloppy reward (this is called *reward hacking*, and we will meet a spectacular example of it later in this report — an arm that “dances” for enormous reward while doing zero useful work).

Unsupervised skill discovery (USD) asks a radical question: what if we give the robot **no reward at all**? Instead we give it a knob — a vector z called the *skill* — and a single, task-free instruction:

*“Whatever you do, do something so different for each setting of the knob
that an observer could tell which setting you were given.”*

That’s it. No definition of digging, walking, or grasping. The hope — borne out in practice — is that in order to be *distinguishably different*, the robot must actually *do* things: crawl left vs. crawl right, jump vs. crouch. Behaviors emerge the way a child’s sandbox repertoire emerges: not because anyone defined “dig,” but because doing distinct things is the game itself. Afterwards, the knob becomes a steering wheel: to make the robot do skill- z -like things, set the knob to z .

Prior to this project, USD had been demonstrated on robots moving *their own bodies* (locomotion) and rigid objects. It had never been shown on **materials** — soil, sand, dough — which is precisely where reward design is hardest and USD would be most valuable. Our testbed: a simulated excavator arm over a bed of ~ 1200 physically simulated soil grains. The question: *can it invent digging?*

2 Mathematical toolkit I: probability, surprise, and information

Everything in this field is built from a small set of probability ideas. We build them here honestly, because the differences between the methods we study *are* differences between these quantities.

2.1 Random variables, distributions, expectation

A *random variable* X is a quantity whose value we are uncertain about — the roll of a die, the position of a soil grain. Its *distribution* $p(x)$ assigns each possible value a probability (or, for continuous quantities, a probability *density*). Two facts we use constantly:

$$\sum_x p(x) = 1 \quad \text{and} \quad \mathbb{E}[f(X)] = \sum_x p(x) f(x),$$

the *expectation*: the average of f over many samples of X , weighted by how likely each value is. When you see $\mathbb{E}[\cdot]$ anywhere in this report, read it as “the average value of the thing inside, in the long run.”

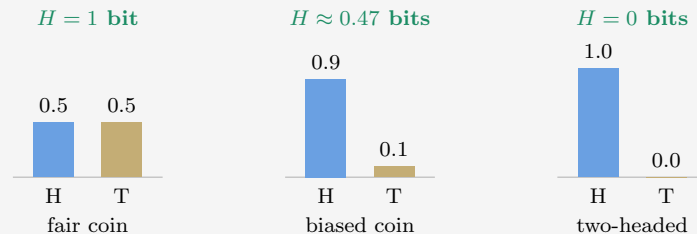
2.2 Entropy: measuring uncertainty

Definition 1 (Entropy). $H(X) = -\sum_x p(x) \log p(x) = \mathbb{E}[-\log p(X)]$.

The quantity $-\log p(x)$ is the *surprise* of seeing x : rare events (small p) are very surprising (large $-\log p$); certain events ($p = 1$) carry zero surprise. Entropy is therefore **average surprise** — how unpredictable X is overall.

Worked example

The case, drawn. Three coins; each bar is the probability of an outcome. Wider spread of probability = more uncertainty = more entropy.



Legend: blue bar = probability of heads; yellow bar = probability of tails; green number = the entropy we are about to compute.

Solved step by step (logs base 2, so the unit is bits):

1. *Fair coin.* Surprise of H: $-\log_2 \frac{1}{2} = 1$. Surprise of T: also 1. Average with weights $\frac{1}{2}, \frac{1}{2}$: $H = \frac{1}{2}(1) + \frac{1}{2}(1) = \mathbf{1 \text{ bit}}$.
2. *Biased coin* ($p_H=0.9$). Surprise of H: $-\log_2 0.9 \approx 0.152$ (common, barely surprising). Surprise of T: $-\log_2 0.1 \approx 3.32$ (rare, very surprising). Weighted average: $H = 0.9(0.152) + 0.1(3.32) \approx \mathbf{0.47 \text{ bits}}$ — the rare outcome is very surprising but rarely happens, so average surprise is *low*.
3. *Two-headed.* Only H can occur, surprise $-\log_2 1 = 0$: $H = \mathbf{0 \text{ bits}}$. Certainty carries no information.

Matching the picture: the flatter the bars, the higher the green number. Entropy peaks when all outcomes are equally likely.

2.3 Conditional entropy and mutual information

$H(X | Y)$ is the average surprise remaining about X *after you are told* Y . If Y tells you a lot about X , little surprise remains.

Definition 2 (Mutual information).

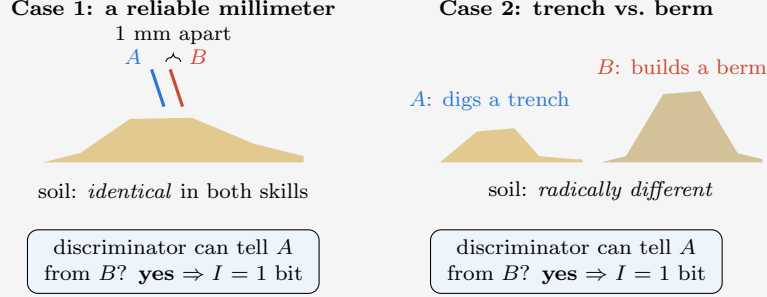
$$I(X; Y) = H(X) - H(X | Y).$$

Intuition

Mutual information is **how much your uncertainty about X drops when you learn Y** — measured in bits. $I = 0$: independent. $I = H(X)$: Y tells you X exactly. Crucially, MI only cares about *distinguishability*, never about *magnitude*: if two skills differ by a soil grain displaced one millimeter, but that millimeter is *reliably* there, MI is just as satisfied as if they had moved a whole dune. Keep this in mind — it is the exact seed of DIAYN's failure on soil.

Worked example

The case, drawn. Two skills, $z \in \{A, B\}$, equally likely. Two scenarios for what the skills actually do:



Legend: yellow = soil; blue/red strokes = where skill A/B leaves the bucket; the boxes give MI's verdict. MI scores both cases identically.

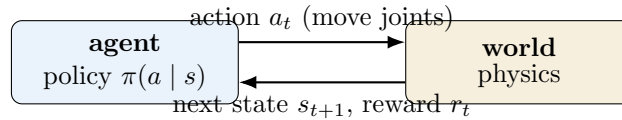
Solved step by step:

- Skills are equally likely, so before observing anything, $H(Z) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$ bit of uncertainty about which skill is running.
- Case 1. The sensor resolves the millimeter, and the millimeter is *reliable* (always there). Seeing the state, you know the skill with certainty: the remaining uncertainty is $H(Z | S) = 0$.
- Therefore $I(S; Z) = H(Z) - H(Z | S) = 1 - 0 = 1$ bit — the *maximum possible* for two skills.
- Case 2. A trench and a berm are also perfectly distinguishable: again $H(Z | S) = 0$, again $I(S; Z) = 1$ bit.
- Compare: $1 = 1$. **MI cannot prefer the trench over the millimeter.** It measures *distinguishability*, which saturates, not *magnitude of change*, which does not. Any method whose reward is MI inherits this indifference — and on soil we later measure exactly this failure (Experiment 6).

3 Mathematical toolkit II: reinforcement learning in one section

3.1 The loop

At each time step t , the agent sees a *state* s_t (for us: arm joint angles, joint speeds, a coarse soil height profile — 35 numbers), picks an *action* a_t (3 numbers: velocity commands for the three arm joints), and the world's physics produces the next state s_{t+1} plus a reward r_t .



Legend: blue box = the learner (a neural network mapping state $\rightarrow$ action). Yellow box = everything it cannot control (the simulator). The loop repeats every 15 ms of simulated time; learning means adjusting the blue box so the stream of r_t grows.

The agent's behavior rule is a *policy* $\pi(a | s)$: a probability distribution over actions given the state. The goal of RL:

$$\max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right], \quad \gamma \in (0, 1),$$

maximize expected *discounted return*. The discount γ (we use 0.99) makes rewards now worth slightly more than rewards later, which keeps the sum finite and encodes mild impatience.

3.2 Value functions and the critic

The *Q-function* $Q^\pi(s, a)$ is the expected return if you take action a in state s and follow π afterwards. It obeys a recursive consistency condition (the Bellman equation):

$$Q^\pi(s, a) = \mathbb{E}[r + \gamma Q^\pi(s', a')], \quad a' \sim \pi(\cdot | s').$$

Modern *actor-critic* methods train two neural networks: a *critic* that estimates Q by minimizing the mismatch between the two sides of this equation, and an *actor* (the policy) that adjusts itself to pick actions the critic scores highly.

3.3 SAC, the workhorse we use everywhere

Soft Actor-Critic (SAC) is a standard, robust actor-critic algorithm with one twist: it maximizes reward *plus* the entropy of the policy,

$$\max_{\pi} \mathbb{E} \left[\sum_t \gamma^t (r_t + \alpha H(\pi(\cdot | s_t))) \right].$$

The entropy bonus keeps the policy exploratory (never prematurely certain), with the temperature α auto-tuned. Implementation details we use: two critics taking the minimum (avoids overestimation), a slowly-updated “target” copy of the critics (stabilizes the bootstrap), and a replay buffer of past transitions that gradient updates are sampled from. Every method compared in this report uses **the exact same SAC machinery** — the methods differ only in where r_t comes from.

3.4 Skill-conditioning

In USD there is no external r_t . Instead: at the start of each episode a skill z is drawn at random from a fixed prior (ours: uniform on the unit sphere in $\mathbb{R}^4$, i.e. a random direction), the policy becomes $\pi(a | s, z)$ — the same network, with z appended to its input — and the reward is *manufactured* by the method itself to make behaviors depend on z . The next section is the story of three ways to manufacture that reward, and what each one gets wrong.

4 The three landmark methods, and the flaw in each

4.1 DIAYN (2019): reward being identifiable

Objective. Maximize the mutual information between states visited and the skill: $I(S; Z) = H(Z) - H(Z | S)$. Since the skill prior is fixed, $H(Z)$ is constant, so the job is to *minimize* $H(Z | S)$ — make the skill easy to guess from the state.

Mechanism. There is a snag: $H(Z | S)$ contains the *posterior* $p(z | s)$ — “given this state, which skill produced it?” — which nobody knows. The fix is the *variational bound*, a two-line derivation worth seeing once. For *any* approximating distribution $q_\varphi(z | s)$ (a neural network with parameters φ):

$$\mathbb{E}[\log p(z | s)] = \mathbb{E}[\log q_\varphi(z | s)] + \underbrace{\mathbb{E}[\log \frac{p(z|s)}{q_\varphi(z|s)}]}_{\text{a KL divergence } \geq 0} \geq \mathbb{E}[\log q_\varphi(z | s)].$$

(The dropped term is a *Kullback-Leibler divergence*, an average log-ratio between two distributions; it is provably never negative, and is zero exactly when q_φ equals the true posterior.) So the

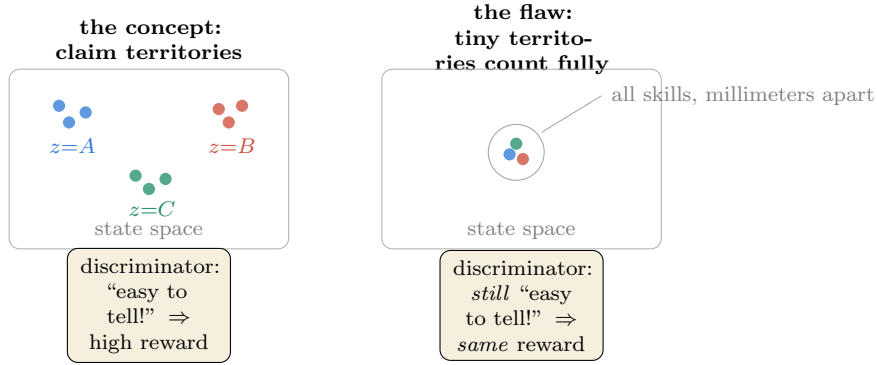
unknowable quantity is *at least* the knowable one: train the discriminator q_φ to guess skills well, and you get an ever-tightening lower bound on the MI. The per-step reward is then

$$r_t = \log q_\varphi(z \mid s_{t+1}) - \log p(z),$$

positive when the discriminator recognizes your skill better than chance.

Intuition

Two players cooperate. The *policy* tries to visit states that scream “I am skill z !”. The *discriminator* learns to hear the scream. Reward is high when the discriminator confidently guesses right. Behaviors must therefore spread out over states — each skill claims its own territory.



Legend: dots = states visited by each skill (one color per skill). Left: skills spread across the state space — discriminator happy. Right: skills differ by a reliable hair’s width — discriminator equally happy, reward equally maximal. Nothing in the objective prefers left over right.

Worked example

DIAYN’s reward, computed — and its ceiling. Two skills, uniform prior $p(z) = \frac{1}{2}$; reward $r = \log_2 q_\varphi(z \mid s) - \log_2 p(z)$ (base 2, so bits).

1. *Early training:* the discriminator is unsure, $q = 0.6$. $r = \log_2 0.6 - \log_2 0.5 = -0.74 - (-1) = +0.26$ bits. Small positive reward: keep separating.
2. *Later:* skills are quite distinguishable, $q = 0.95$. $r = \log_2 0.95 + 1 = +0.93$ bits.
3. *The ceiling:* even a perfect discriminator gives $r = \log_2 1 + 1 = 1$ bit — and no more, ever. The reward can never exceed $\log_2 \frac{1}{p(z)}$.
4. *The flaw, numerically:* the millimeter-skill from the previous worked example already achieves $q \approx 1$, i.e. reward $\approx$ the ceiling. Digging a trench instead would raise the reward by ≈ 0.00 bits. **Gradient toward more change: zero.** That is saturation — and on soil we measured the discriminator at 0.98 cosine within the first few thousand steps, reward flat ever after (Figure 7, middle).

The flaw

Identifiable $\neq$ substantial. A skill that tilts one joint by one degree *and holds it* is perfectly identifiable. MI rewards **distinguishability, not magnitude, and not continued change**: once states are separable, the game is won and nothing pushes further. On our soil domain we measured exactly this: the discriminator reached 0.98 accuracy while soil movement stayed at the ambient noise floor (Section 7.6). We call this failure mode **saturation**.

4.2 DADS (2020): reward being predictable

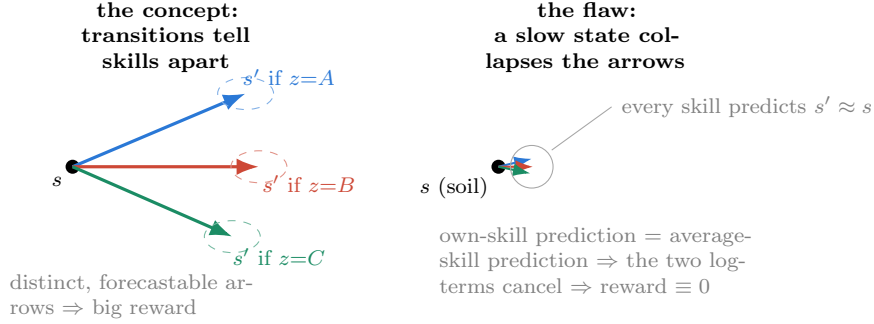
Objective. Skills should have *modelable consequences*. Learn a skill-conditioned dynamics model $q_\theta(s' | s, z)$ (“if I am doing skill z in state s , where do I land?”), and reward transitions that this model predicts well *under your own skill* but that look unlike other skills’ transitions:

$$r_t = \log q_\theta(s_{t+1} | s_t, z) - \log \frac{1}{L} \sum_{l=1}^L q_\theta(s_{t+1} | s_t, z_l), \quad z_l \sim p(z).$$

The second term is the average prediction quality under L *randomly drawn alternative skills*: your transition should be *yours* — predictable given z , surprising given anything else.

Intuition

DIAYN asks “can an observer tell skills apart by *where you are*?” DADS asks “can an observer tell skills apart by *how you move*?” — and additionally demands that each skill’s motion be regular enough to forecast, which is what later makes planning with skills possible.



Legend: black dot = current state; colored arrows = each skill’s predicted next state (dashed ellipse = prediction spread). Left, locomotion: skills produce different, forecastable futures. Right, soil: the material barely moves per step, so all three arrows land on the same spot — there is nothing to be “more predictable than the others” about.

Worked example

DADS’s reward, computed twice. Log-densities of a Gaussian prediction model: $\log q \propto -\text{error}^2/(2\sigma^2)$; take $\sigma = 1$ and L alternative skills. Reward $r = \log q(s' | s, z) - \log \frac{1}{L} \sum_l q(s' | s, z_l)$.

1. *Fast state (locomotion-like).* Your own skill predicts the transition well: error 0.1 $\Rightarrow \log q_z = -0.005$. Other skills predict it badly: error 1.0 $\Rightarrow \log q_{z_l} = -0.5$ each. Reward $r = -0.005 - (-0.5) = +\mathbf{0.495}$. Positive: your motion is *yours*. Learning proceeds.
2. *Slow state (soil).* The soil barely changes: $s' - s \approx 0$. Every skill’s model predicts “no change” perfectly: own error ≈ 0 and alternatives’ error ≈ 0 . Reward $r = 0 - 0 = \mathbf{0.000}$. Not small — *exactly* zero, because the two terms are the same number.
3. There is no gradient in step 2 and never will be: to escape, the policy would need to *already* be moving soil in skill-distinct ways — which is precisely what the zero reward fails to teach. Chicken and egg; the measured result is a flatline at 0.000 for 200,000 steps (Figure 7, bottom).

The flaw

The signal lives entirely in the **per-step change** $s' - s$. Materials are *slow*: most steps, the soil barely changes. Then $s' \approx s$ is trivially predictable *under every skill*, the two log-terms cancel, and the reward is identically zero — not small, *zero*. We measured DADS-with-soil rewards at 0.000 for an entire 200,000-step run (Figure 7, bottom panel). We call this failure mode **starvation**.

4.3 METRA (2024): reward going far

Objective. Learn a map $\phi: \text{states} \rightarrow \mathbb{R}^D$ (the *representation*) and make each skill travel along its own direction in that latent space:

$$\max_{\pi, \phi} \mathbb{E}[(\phi(s_{t+1}) - \phi(s_t))^\top z] \quad \text{subject to} \quad \|\phi(s) - \phi(s')\| \leq d(s, s'), \quad (1)$$

where z is a unit vector and d is some notion of distance between states.

Why the constraint must exist. Without it there is a fatal cheat: ϕ could simply *stretch* — multiply itself by 1000 — and every reward would grow 1000-fold with no behavioral change. The constraint pins the latent space to reality: *one unit of latent motion must be backed by at least one unit of real motion*, as measured by d . METRA’s choice of d is the *temporal* distance (roughly: adjacent steps are at distance ≤ 1).

How a constraint is enforced in practice (Lagrangian duality, gently). You cannot tell a neural network “obey this inequality.” Instead you convert the constrained problem into an unconstrained one with a *price*. Define the penalized objective

$$\mathcal{L}(\phi, \lambda) = \underbrace{\mathbb{E}[(\phi(s') - \phi(s))^\top z]}_{\text{reward}} + \lambda \cdot \underbrace{\mathbb{E}[\min(\epsilon, d^2 - \|\phi(s) - \phi(s')\|^2)]}_{\text{constraint slack (positive = obeyed)}}.$$

$\lambda \geq 0$ is a learnable *price of violation*: while the constraint is violated (slack negative) the price rises, punishing ϕ harder; when obeyed, the price falls. The system self-tunes to run right at the boundary — which is exactly where you want it, since reward grows with latent distance and latent distance is capped by real distance. (We watch λ throughout training as a health signal; ours stabilizes around $\lambda \approx 30$.)

Intuition

Think of λ as a *fine* set by a regulator. The network wants to stretch its map (more stretch = more reward); stretching past what physical distance justifies incurs the fine. If networks keep cheating, the regulator raises the fine until they stop; if nobody cheats, the fine decays so it doesn’t distort honest behavior. At equilibrium the map is stretched *exactly* to the legal limit — taut like a drumhead — which is what makes latent distances meaningful measurements of real distances.

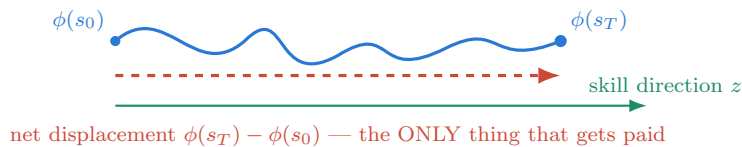
The telescoping property — the single most load-bearing equation in this report. Sum the per-step rewards over an episode:

$$\sum_{t=0}^{T-1} (\phi(s_{t+1}) - \phi(s_t))^\top z = (\phi(s_T) - \phi(s_0))^\top z. \quad (2)$$

To see it, write out three steps and watch the middle terms annihilate:

$$(\phi_1 - \phi_0)^\top z + (\phi_2 - \phi_1)^\top z + (\phi_3 - \phi_2)^\top z = (\cancel{\phi_1} - \phi_0 + \cancel{\phi_2} - \cancel{\phi_1} + \phi_3 - \cancel{\phi_2})^\top z = (\phi_3 - \phi_0)^\top z.$$

Every intermediate position appears once with $+$ and once with $-$ (a collapsing telescope). The total reward of an episode depends *only on where you ended relative to where you started*: wiggling back and forth earns nothing net; a step backward *costs* exactly what the forward step earned.

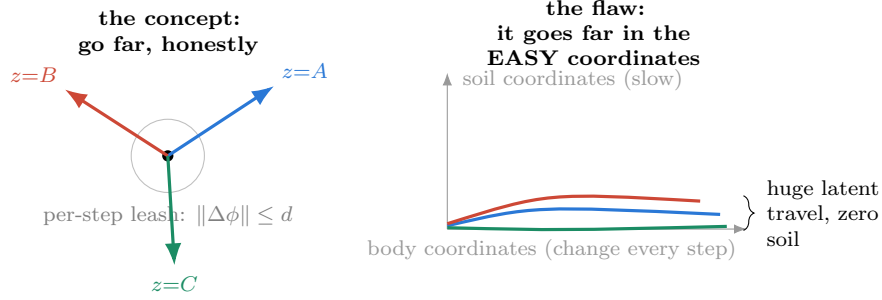


Legend: blue curve = the wandering latent trajectory over one episode (all its wiggles cancel in the reward sum). Red dashed arrow = the net start-to-end displacement; the episode’s total reward is this arrow’s length along the green skill direction z .

This is why we will call distance-maximizing objectives *cumulative* — and it is the property that will single-handedly decide the frontier comparison in Section 7.

Intuition

DIAYN pays you for *being recognizable*. DADS pays you for *moving recognizably*. METRA pays you for **net displacement** — and, through the constraint, latent displacement is backed by real-world distance. You cannot micro-cheat; you must actually go somewhere and stay there.



Legend. Left: from the current latent position (black dot), each skill pushes in its own direction; the gray circle is the per-step constraint — one unit of latent motion must be backed by real motion. Right: when ϕ sees the full state, the cheap way to “go far” is along the body axes, which move at every step; all skills race horizontally while the soil axis stays untouched.

Worked example

METRA’s reward, computed — honest version and exploited version. Take skill $z = +1$ in a 1-D latent for arithmetic’s sake.

1. *Telescoping, with numbers.* Suppose over four steps $\phi = 0 \rightarrow 0.3 \rightarrow 0.2 \rightarrow 0.6$. Per-step rewards: $+0.3, -0.1, +0.4$. Sum = **0.6** = $\phi_{\text{end}} - \phi_{\text{start}}$. The backward wiggle (-0.1) was *charged*, not ignored: only net progress pays.
2. *Our constraint ($d = W_2$ on soil).* Step moves 0.35 m of soil transport $\Rightarrow |\Delta\phi| \leq 0.35$: latent progress is capped by *physical work*. Arm-waves move 0.000 m of soil $\Rightarrow |\Delta\phi| \leq 0 \Rightarrow$ reward for arm-waving = **0**, by construction.
3. *Vanilla METRA’s constraint (temporal, $d \equiv 1$ per step).* Any adjacent states are “distance 1” apart — including two states that differ only in arm pose. So an arm-wave may earn up to 1 latent unit *per step*: over a 200-step episode, up to **200** units of latent travel with zero soil moved. Measured: its latent fan spans ± 150 (Figure 9) over untouched terrain — within 25% of this back-of-envelope ceiling.

The flaw

Go far — *in which part of the state?* The state contains the robot’s own body, and the body is the easiest thing in the world to move far. Every one of these objectives, fed the full state, feasts on proprioception. In a manipulation scene the result is an agent that develops elaborate arm choreography and never touches the object — documented by the CSD authors (2023) as “completely ignoring the object,” and reproduced by us, quantified, on soil (Section 7.3).

5 Mathematical toolkit III: optimal transport

Our original idea needs one more mathematical instrument — a beautiful one. But first, a decoder for the three “distances” this report keeps comparing.

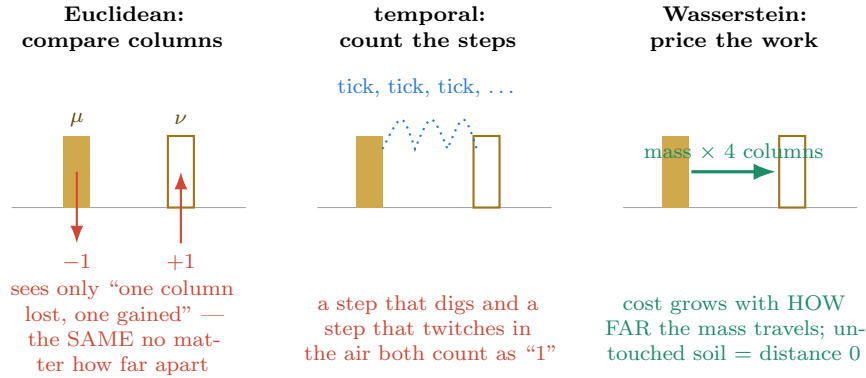
5.1 Three rulers for the same question

Everywhere in skill discovery, some quantity $d(s, s')$ must answer: *how different are two states?* The three candidate rulers give radically different answers to the same pair of soil states — and each has a blind spot the experiments will exploit.

Euclidean distance: compare the lists, entry by entry. A soil state is a list of numbers (mass per column). Euclidean distance subtracts the lists entry-wise, squares, sums, roots: $d_{\text{euc}} = \sqrt{\sum_i (\mu_i - \nu_i)^2}$. It asks: *“how much does each column differ?”* — and nothing else. **Blind spot: geometry.** Take a pile occupying one column, and shift it. Shift by one column or by fifty: in both cases the old column lost everything and some new column gained everything, so *the entry-wise difference is identical*. With $\mu = (1, 0, 0, \dots)$: shifting to column 2 gives $d_{\text{euc}} = \sqrt{1^2 + 1^2} = 1.41$; shifting to column 50 gives *the same* 1.41. Near rearrangement and massive rearrangement — indistinguishable.

Temporal distance: count the steps. METRA’s ruler. The distance between two states is (roughly) *the number of environment steps needed to get from one to the other*; in practice, states one step apart are at distance ≤ 1 . It asks: *“how long does it take?”* **Blind spot: content.** One time-step in which the bucket drags 100 kg of soil, and one time-step in which the arm twitches in the air, are the *same* distance: 1. The ruler prices effort in seconds, not in consequences — which is exactly the loophole the arm-dancing exploit drives through (one latent unit per step, 200 steps, zero soil).

Wasserstein distance: price the earth-moving. The physical ruler, developed in the rest of this section. It asks: *“how much work — mass $\times$ distance — would a perfect bulldozer need to turn one pile into the other?”* Shift a pile one column: cheap. Shift it fifty columns: fifty times the work. Wave the arm: the pile is unchanged, distance zero. Its answers are denominated in physical units. **Blind spot** (found by our experiments, Section 7): it prices soil work correctly, but as a *learning signal* it can be too faint to bootstrap from — correctness and trainability are different virtues.



Legend: in every panel, the filled block is the pile now (μ), the hollow block is the target pile (ν), four columns to the right. Three rulers, three verdicts on the same pair: Euclidean sees “two columns changed” (blind to the four-column gap); temporal sees “however many ticks the machine needs” (blind to what each tick did); Wasserstein sees “this much mass moved this far” — the physical answer.

Intuition

Same question, three currencies. Euclidean pays in *pixels changed*, temporal pays in *seconds spent*, Wasserstein pays in *tons-moved-times-meters*. Our original hypothesis was that the third currency — the only physical one — would make skills better. The experiments' surprise: within the working recipe, all three currencies buy the same skills (Experiment 2), because the learned map ends up physically calibrated *no matter which currency trained it*. The currency that matters is the one the *conditioning* chooses: what ϕ is allowed to see.

5.2 The earth mover's problem

You have a pile of sand shaped like μ and want a pile shaped like ν (same total mass). Moving one kilogram of sand a distance of one meter costs one unit of work. **What is the cheapest total work?** That minimal cost is a *distance between the two shapes*: small if ν is a slight rearrangement of μ , large if mass must travel far. This is the *Wasserstein distance*, also called the *earth mover's distance* — and for us the name is literal: **our robot is the earth mover**, and its job description is this exact integral.

5.3 The formal definition

Definition 3 (Kantorovich formulation of W_2). *For distributions μ, ν on $\mathbb{R}^n$:*

$$W_2(\mu, \nu) = \left(\min_{\gamma \in \Pi(\mu, \nu)} \int \|x - y\|^2 d\gamma(x, y) \right)^{1/2},$$

where $\Pi(\mu, \nu)$ is the set of transport plans: joint distributions $\gamma(x, y)$ whose marginals are μ and ν . Read $\gamma(x, y)$ as “how much mass I ship from location x to location y ”; the marginal conditions say you ship away exactly the pile you have and deliver exactly the pile you want.

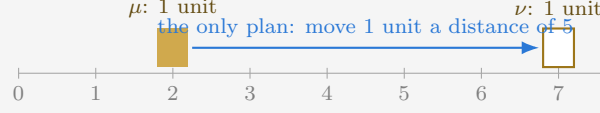
Why squared distance (W_2 rather than W_1)? Squared costs favor many short moves over one long move, give the distance Riemannian-geometry structure (geodesics, barycenters), and connect to fluid dynamics (Benamou–Brenier: W_2 is the minimum kinetic energy of a flow morphing μ into ν). For this report it suffices that W_2 is the standard choice with the richest theory.

Contrast with the *Euclidean* distance between the piles' height profiles, $\|\mu - \nu\|_2$: that compares the piles **column by column**, and is blind to geometry — a pile shifted 10 cm and the same pile shifted 5 m look equally “different,” because in both cases each column changed. Wasserstein knows the second rearrangement is fifty times more work.

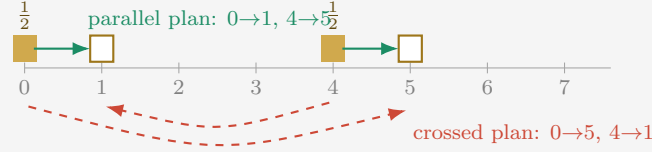
Worked example

The case, drawn. Sand as blocks on a number line; arrows are transport plans (“ship this block there”).

Part 1: one block



Part 2: split blocks — two competing plans



Legend: filled blocks = where the sand IS (μ); hollow blocks = where it *SHOULD END UP* (ν); each arrow ships the fraction written on the block. Green solid = the non-crossing plan; red dashed = the crossing plan. We now price both.

Solved step by step (recall: cost = mass $\times$ squared distance, then $W_2 = \sqrt{\text{min total cost}}$):

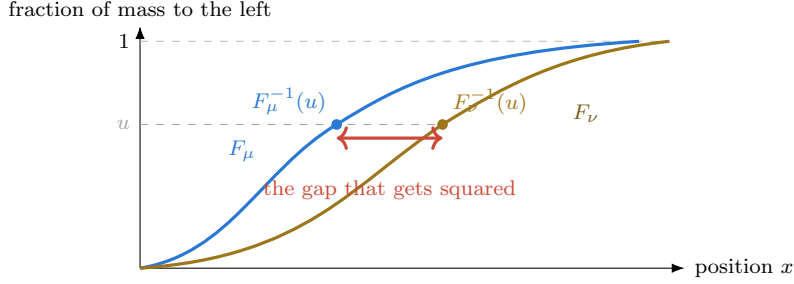
1. *Part 1.* One unit travels $7 - 2 = 5$. Cost = $1 \cdot 5^2 = 25$. Only one plan exists, so $W_2 = \sqrt{25} = 5$ — exactly the distance the mass traveled. Sanity restored: the metric *is* “how far did the sand move.”
2. *Part 2, green (parallel) plan:* $\frac{1}{2}$ unit goes $0 \rightarrow 1$ (distance 1) and $\frac{1}{2}$ unit goes $4 \rightarrow 5$ (distance 1). Cost = $\frac{1}{2}(1^2) + \frac{1}{2}(1^2) = 1$.
3. *Part 2, red (crossed) plan:* $\frac{1}{2}$ unit goes $0 \rightarrow 5$ (distance 5) and $\frac{1}{2}$ unit goes $4 \rightarrow 1$ (distance 3). Cost = $\frac{1}{2}(25) + \frac{1}{2}(9) = 17$.
4. The minimum over plans picks green: $W_2 = \sqrt{1} = 1$, seventeen times cheaper than red. Notice *why* green wins: its arrows never cross. Uncrossing any crossing always shortens both trips — which is the reason the 1D optimum is simply “match the blocks in left-to-right order,” i.e. match quantiles, i.e. Equation (3).

5.4 The one-dimensional miracle

In general, computing W_2 means solving an optimization (linear programming, or the entropic/Sinkhorn approximation). But in **one dimension** the answer is closed-form. The reason: on a line, an optimal plan never lets two mass streams *cross* (if stream A goes left-to-right past stream B going right-to-left, uncrossing them shortens both trips). No crossing means the cheapest plan matches mass *in order*: the leftmost 1% of μ goes to the leftmost 1% of ν , and so on. “The u -th quantile maps to the u -th quantile”:

$$W_2^2(\mu, \nu) = \int_0^1 (F_\mu^{-1}(u) - F_\nu^{-1}(u))^2 du, \quad (3)$$

where F^{-1} is the quantile function (the inverse of the cumulative distribution function: $F_\mu^{-1}(0.5)$ is the median position of the mass). Computationally: two cumulative sums, one binary search, one mean of squared differences — microseconds on a GPU, exact.



Legend: each curve climbs from 0 to 1 as it sweeps left to right, accumulating its pile’s mass (blue = pile μ , yellow = pile ν). Pick any mass level u (dashed): the **horizontal** gap between the curves at that level is how far the u -th “slice” of sand must travel. W_2^2 is the average of that squared gap over all levels $u \in [0, 1]$ — Equation (3) is literally this picture, integrated.

Our soil bed, summarized as *mass per vertical column* over the horizontal axis (64 bins), is exactly such a 1D distribution, and soil mass is conserved during an episode — so this formula gives us the **true physical transport cost between any two soil states, thousands of times per second, during training**. (For genuinely 2D mass fields there is *sliced* W_2 : average the 1D formula over many random projection directions. We use it in Experiment 5.)

6 Our method: TMSD

6.1 The two ingredients

Take METRA’s template (1) and change two things:

Ingredient 1 — a physical ground metric. Let μ_s denote the soil mass distribution in state s , and set

$$d(s, s') = W_2(\mu_s, \mu_{s'}).$$

The Lipschitz constraint becomes: *one unit of latent motion must be backed by one unit of physical mass transport* (tons moved $\times$ meters). Skills, to earn reward, must differ by how they *move soil*.

Ingredient 2 — external-state conditioning. The representation ϕ receives **only** the soil measure μ — never a joint angle, never a velocity. Then for any behavior that only moves the arm, $\phi(\mu') = \phi(\mu)$ and the reward is *exactly zero by construction*. Arm-dancing is not discouraged; it is **unpaid**.

The full objective:

$$\max_{\pi, \phi} \mathbb{E}[(\phi(\mu_{t+1}) - \phi(\mu_t))^\top z] \quad \text{s.t.} \quad \|\phi(\mu) - \phi(\mu')\| \leq W_2(\mu, \mu'), \quad (4)$$

optimized with the Lagrangian dual described earlier (with a *relative* slack cap, because W_2 steps span orders of magnitude, unlike METRA’s constant temporal distance), and SAC underneath. The intrinsic reward is recomputed with the *current* ϕ at every gradient step, since ϕ keeps improving and stale rewards would anchor the policy to an obsolete map.

6.2 What we believed, and what we pre-registered

Going in, we believed Ingredient 1 (the transport metric) was the scientific contribution and Ingredient 2 was plumbing. Following good practice, we pre-registered the ablation that could falsify this (swap W_2 for Euclidean and temporal metrics) *and* the pivot rule if it failed. It failed. The report below is the honest record — and the replacement finding is stronger than the original bet.

6.3 The testbed

Custom 2D discrete-element (DEM) simulation: ~ 1200 circular soil grains with Hertzian contact forces, Coulomb friction, rolling resistance and cohesion; a three-joint (boom/stick/bucket) hydraulic arm with realistic torque and speed limits, base fixed at $x=2\text{m}$ in a 12.8m domain; CUDA-accelerated. Agent observation (35-dim): joint angles/velocities, wrist pose, contact forces, payload mass, 16-bin soil heightfield. Action (3-dim): joint velocity commands. The soil measure μ for ϕ : a 64-bin, mass-weighted, normalized histogram over x . Skills: z uniform on the unit sphere in $\mathbb{R}^4$. Episodes: 200 steps. All methods trained with identical budgets (200k environment steps = 200k gradient updates, $\sim 1.5\text{--}2.5\text{h}$ per run on one RTX 5080).

6.4 How the state is represented: four layers

It is worth being very explicit about *who sees what*, because the project’s central finding lives entirely in this split. Figure 1 shows all four layers for one real state:

1. **Ground truth** (simulation only): ~ 1200 particles $\times$ (position, velocity, radius, mass) plus the arm — thousands of dimensions nobody observes directly.
2. **The policy’s observation** (35 numbers): joint angles and velocities, wrist pose, contact forces, payload, and a coarse 16-bin soil heightfield. The *policy* is allowed to know its own body — it needs to, to control it.
3. **The external measure μ** (64 numbers): soil mass per column, normalized to sum to 1, *zero robot information*. This — and only this — is what the objective’s representation ϕ ever receives. Note the asymmetry with layer 2: the policy’s soil view is a coarse *height* field; ϕ ’s is a finer *mass* distribution — the physically meaningful object for transport.
4. **The latent $\phi(\mu) \in \mathbb{R}^4$** : a small MLP trained under the Lipschitz constraint. Skills are directions here, goals are points here, and steering is arithmetic here.

7 The experiments

Each experiment follows the same template: *the problem* it answers, *the test*, *the result* (with the actual data), and *the insight* that determined what we did next.

7.1 Experiment 1 — Does anything emerge at all?

Problem. No one has run USD on granular media. Perhaps nothing happens: contact-rich digging is hard even for rewarded RL.

Test. TMSD as in (4), 500k steps, fixed initial terrain, skill dimension 2 (a pilot: small and interpretable).

Result. Skills emerge that *dig*: scoop into the bed, drag material, build spoil piles — reward-free. By late training the policy moves nearly $10\times$ the ambient soil motion per episode.

Insight. The skill coordinate is physically interpretable: the sign of z_1 equals the direction the soil’s center of mass shifts (measured ± 0.10 to 0.16m per episode). One caveat appeared: with the fixed terrain and $\dim z = 2$, only ≈ 3 genuinely distinct behaviors emerged (hold / push-left / push-right around one spot) — the latent capacity bounds how many physical axes the skills can span. This motivated $\dim z = 4$ plus terrain randomization (a random smooth height envelope per episode) for every subsequent experiment.

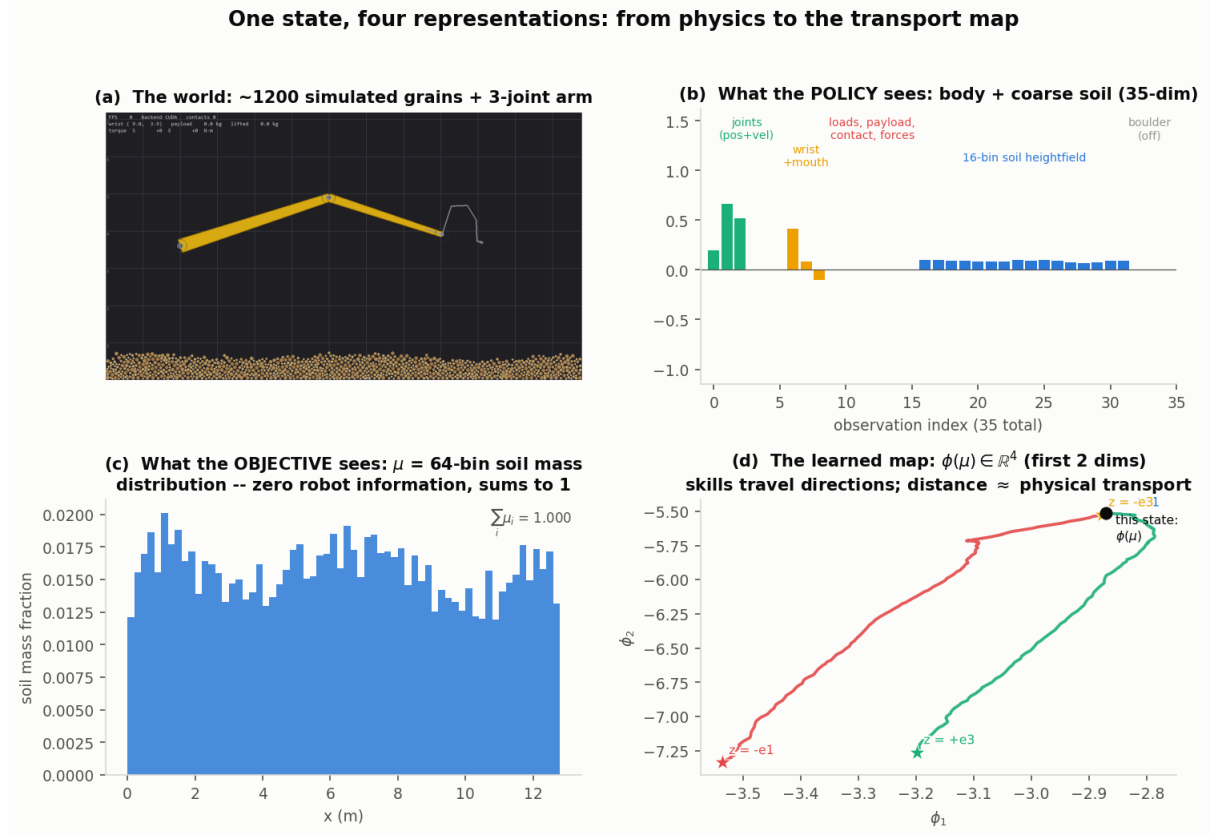


Figure 1: One state, four representations, all real data through the trained model. (a) the simulated world; (b) the 35 numbers the policy acts on, color-grouped; (c) the soil mass distribution μ — the objective’s entire universe, summing to exactly 1; (d) the state’s position on the learned latent map, with four skill episodes traced for context (star = terminal). Note in (d) that one direction ($+e_1$) barely moves from the start: a *dead direction* on this terrain, honestly visible.

7.2 Experiment 2 — Is the transport metric doing the work?

Problem. Our headline hypothesis. If W_2 is the load-bearing ingredient, replacing it should hurt.

Test. Identical recipe, three seeds each, swapping only d in the constraint of (4): (a) W_2 (ours); (b) *Euclidean* distance between the 64-bin histograms (geometry-blind); (c) *temporal* $d \equiv 1$ (METRA’s original). All evaluated by one frozen protocol: coverage = mean pairwise W_2 between terminal soil states over 8 fixed skills $\times$ 3 fixed terrains (the protocol was frozen *before* results were seen, so the ruler could not be chosen to flatter anyone).

Result.

ground metric	W_2 (ours)	Euclidean	temporal (METRA)
coverage (m), 3 seeds	0.326 ± 0.029	0.351 ± 0.030	0.290 ± 0.077

A three-way statistical tie. **Our hypothesis is falsified.**

Insight — why the metric cannot matter here. We measured, for every trained model, the correlation between latent distance $\|\phi(\mu_a) - \phi(\mu_b)\|$ and physical $W_2(\mu_a, \mu_b)$ over thousands of visited state pairs. *Every* model — including those trained with the Euclidean and temporal constraints — came out at ≈ 0.99 (Pearson and Spearman). The latent maps all became near-isometries of the physical transport geometry **without being asked**. Explanation: the set of soil states reachable in a 200-step episode is effectively low-dimensional (dig more/less, left/right, near/far). Along such a thin manifold, all reasonable metrics are monotone re-parameterizations of one another — they can only disagree on state pairs the policy never visits. *Geometry is emergent*

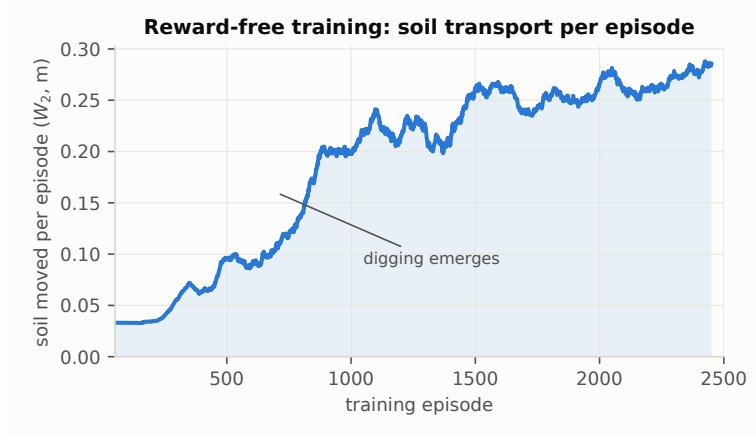


Figure 2: Soil transported per episode (W_2 between the episode’s initial and final soil states) over reward-free training. Rolling mean over 100 episodes. The rise from ≈ 0.03 m (ambient settling — what the soil does with a motionless arm) to ≈ 0.28 m is digging being *invented*.

and free; the metric choice is not where the leverage is.

7.3 Experiment 3 — Then what IS doing the work?

Problem. If not the metric, perhaps the conditioning: ϕ seeing only soil.

Test. Identical run, but ϕ receives the *full* 35-dim observation (proprioception + soil). This is precisely vanilla METRA. Three seeds.

Result. Coverage 0.323 ± 0.028 (soil-only, pooled over metrics, 9 runs) vs. 0.039 ± 0.036 (full-state, 3 runs): an **8 $\times$ effect**, and the full-state failure is qualitative — 7 of 8 skills never touch the soil. Watching it is comic: the arm performs elaborate, skill-distinct choreography above an untouched bed.

Insight. *What you show the discriminator determines what the skills are about.* The ingredient we had treated as plumbing is the load-bearing wall. This reframing — the discriminator’s input as a **task-specification language** (“point the objective’s attention at the subsystem you care about”) — became the project’s central claim.

7.4 Experiment 4 — The metric’s second chance (2D measure)

Problem. Maybe Experiment 2’s tie was an artifact of the crude 1D measure: a deep narrow hole and a wide shallow one have identical column sums, so *no* 1D metric can tell them apart. On a full 2D mass grid (48×24 bins over x, y), transport structure could finally matter.

Test. Rerun the metric ablation with 2D measures: sliced- W_2 (Section 5.4) vs. Euclidean vs. temporal, same recipe.

Result. Sliced- W_2 coverage: 0.044. Euclidean-2D: 0.245. Temporal-2D: 0.214. The transport metric is not merely unhelpful here — it is **5 $\times$ worse, on its own preferred yardstick**.

Insight (from training telemetry, not speculation). Under the SW_2 constraint the representation extracted only $\sim 7\%$ of the reward its constraint budget allowed (the Euclidean control: $\sim 45\%$). The reward signal never grew teeth; the policy never received a usable gradient; behavior stayed at ambient. Mechanism: *optimization starvation*, not mis-valued behavior. The honest summary of Experiments 2+4: **the physical transport metric is conclusively not the source of skill diversity — in the rich regime it can even prevent learning**. What survives of W_2 : rewards with physical units (meters of mass transport), i.e. *calibration*, useful for the goal-reaching layer below.

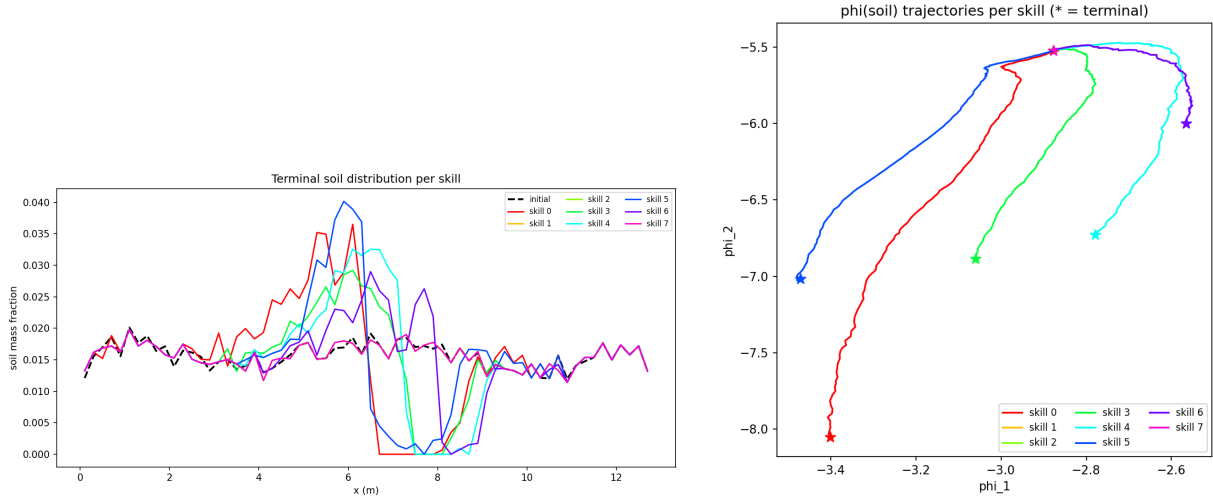


Figure 3: **Left:** terminal soil profiles for 8 skill directions vs. the initial terrain (dashed): different skills excavate and deposit at different locations. **Right:** the latent representation at work — each curve is $\phi(\mu_t)$ over one episode, colored by skill ($\star$ = terminal). The fan structure is the objective (4) realized: each skill travels its own latent direction, and by the constraint, latent distance $\approx$ physical transport. This latent space is the robot’s learned “map of soil states.”

7.5 Experiment 5 — Composing skills into goals (zero-shot shaping)

Problem. Diverse skills are means, not ends. Can they be *aimed*? Given a target soil profile μ^* (a trench, a berm), can the trained system reach it with **no further training**?

Test. The zero-shot steering rule: every 10 steps, command the skill

$$z = \frac{\phi(\mu^*) - \phi(\mu_{\text{now}})}{\|\phi(\mu^*) - \phi(\mu_{\text{now}})\|}$$

— literally “point the skill-knob at the goal’s location on the latent map.” This works only if the latent map is geometrically faithful (which Experiment 2’s isometry result guarantees).

Result. Across 21 targets and 9 checkpoints: **39–63% of the distance to goal removed, zero-shot**; 79–94% of control intervals decrease W_2 *monotonically* — an empirical *contraction certificate* (each skill application provably-in-practice brings the soil closer). In the executable demo, the order “excavate $x \in [7, 9]$, dump at $x \in [4.5, 6.5]$ ” removes **79% of the dig zone’s soil** and fills the dump zone to within 2.6 points of target (Figure 6).

model (seed)	goal improvement	monotone intervals	latent–physical corr.
W_2 (s0 / s1 / s2)	58/45/39 %	91/82/84 %	0.99/0.99/0.99
Euclid (s0 / s1 / s2)	63/54/47 %	94/91/79 %	0.99/0.99/0.99
temporal (s0 / s1 / s2)	63/44/54 %	92/85/91 %	0.99/0.98/0.98

The last column is Experiment 2’s isometry result, per seed: every latent map, whatever its training metric, measures physical transport at correlation ≈ 0.99 — which is precisely why pointing z at the goal’s latent works for all of them, and why none of them steers better than another.

Insight. The plateau $r(\varepsilon)$ is not noise — it is the theoretically expected residual: skills ε -cover the directions of transport, so greedy composition contracts until goals are finer than the coarsest available move, then stalls. This number returns, decisively, in Experiment 8.

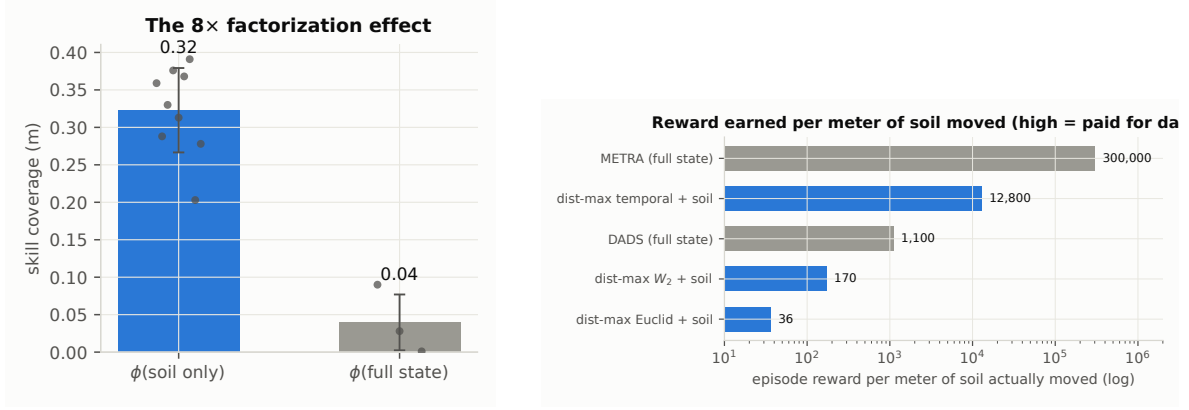


Figure 4: **Left:** the factorization effect — dots are individual runs (9 soil-conditioned across all metrics; 3 full-state). **Right:** reward collected per meter of soil actually moved (log scale): a “lie detector” for objectives. Full-state models earn $\sim 300,000$ reward-units per meter — they are being paid for something other than soil. The W_2 -constrained model’s ~ 170 is the honest number: its rewards are denominated in physical transport.

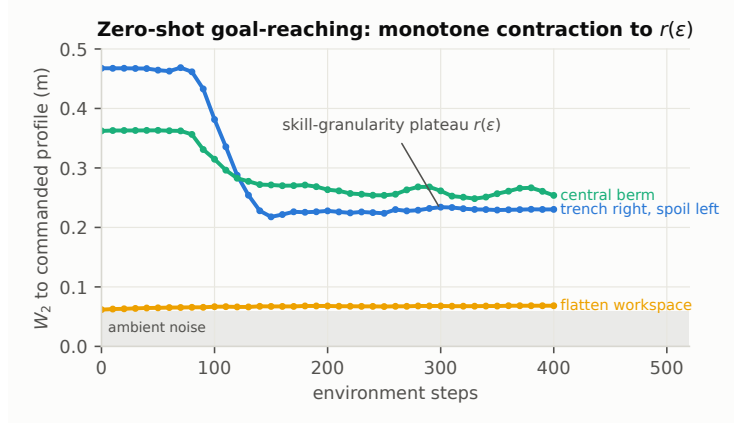


Figure 5: W_2 -to-goal during zero-shot steering, three commanded targets. The trench command halves the distance (+51%); the berm +28%; the “flatten” target starts *below* the resolvable scale and correctly does almost nothing. Note the plateau: contraction stops at $r(\epsilon) \approx 0.2$ m — the *skill granularity radius*.

7.6 Experiment 6 — DIAYN on soil (the MI 2×2)

Problem. Is external conditioning *alone* sufficient? Then an MI objective with soil-only inputs should also work.

Test. Continuous-DIAYN (discriminator predicts z from the state; reward = cosine agreement), crossed with both conditionings, 3 seeds per cell, identical recipe.

Result. Both MI cells fail: coverage 0.021 ± 0.015 (soil-only) and 0.031 ± 0.028 (full-state) — ambient. The mechanism is visible in training curves (Figure 7, middle): the discriminator reaches 0.98 cosine accuracy *almost immediately* — microscopic, static grain differences suffice to identify the skill — and thereafter nothing demands further change. **Saturation**, exactly as the theory of Section 4.1 predicts, now measured on a material.

Insight. Conditioning is necessary but *not* sufficient. The objective family matters too — MI-style identifiability cannot power material manipulation.

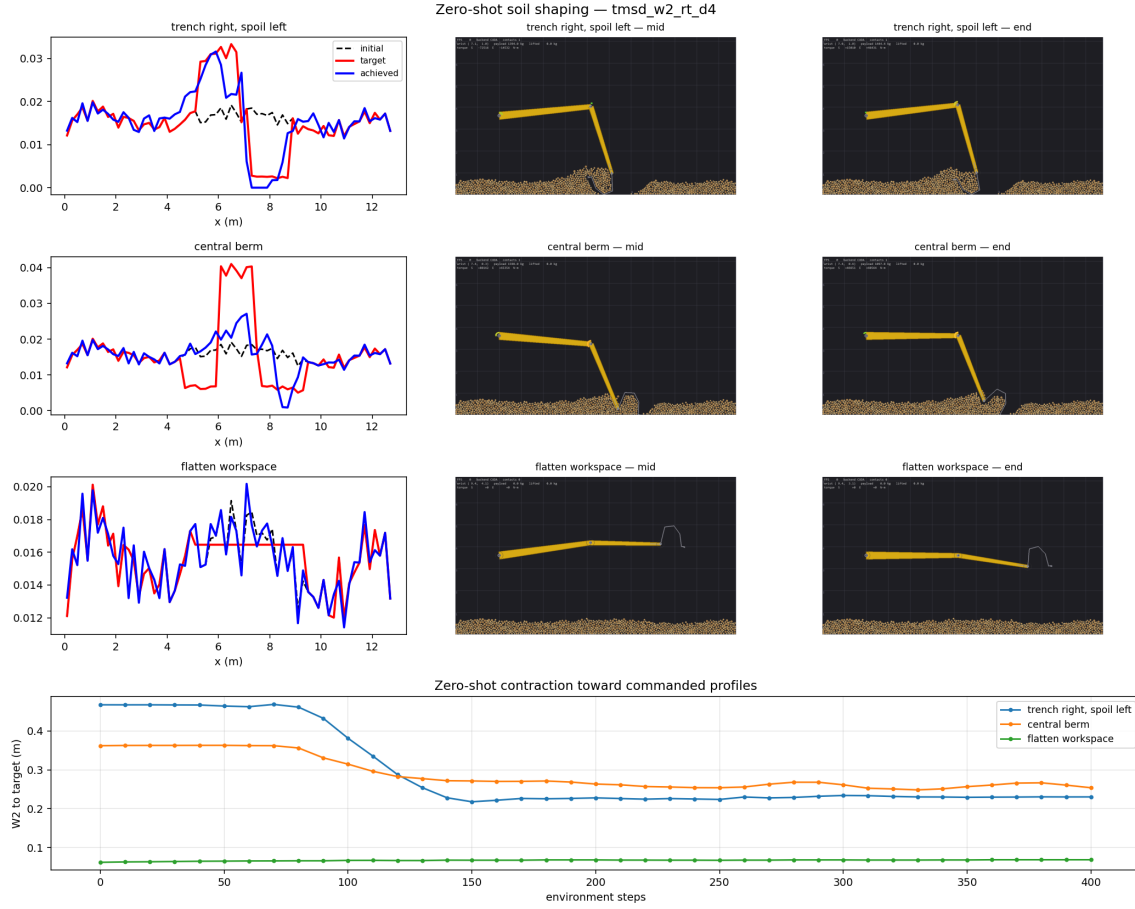


Figure 6: The zero-shot shaping demo in full: per-target profiles (initial / target / achieved), simulator frames mid- and end-execution, and the contraction curves. No reward, no planner, no goal-conditioned training — composition of reward-free skills via latent steering.

7.7 Experiment 7 — DADS on soil

Test. Faithful DADS (skill-dynamics model + predictability-advantage reward, $L=16$ alternatives), both conditionings, 2 seeds per cell.

Result. Soil-conditioned DADS: coverage 0.008 ± 0.008 — statistically zero — with per-step reward measured at **0.000 for the entire run** (Figure 7, bottom). **Starvation**, exactly as Section 4.2 predicts: soil’s per-step deltas are almost always nil, hence equally predictable under every skill, hence zero advantage. Full-state DADS: 0.130 ± 0.073 — one seed reached 0.203 through arm-dynamics skills that *incidentally* contact the bed (its sibling seed: 0.057); highest variance in the study and still $2.5\times$ below the working family.

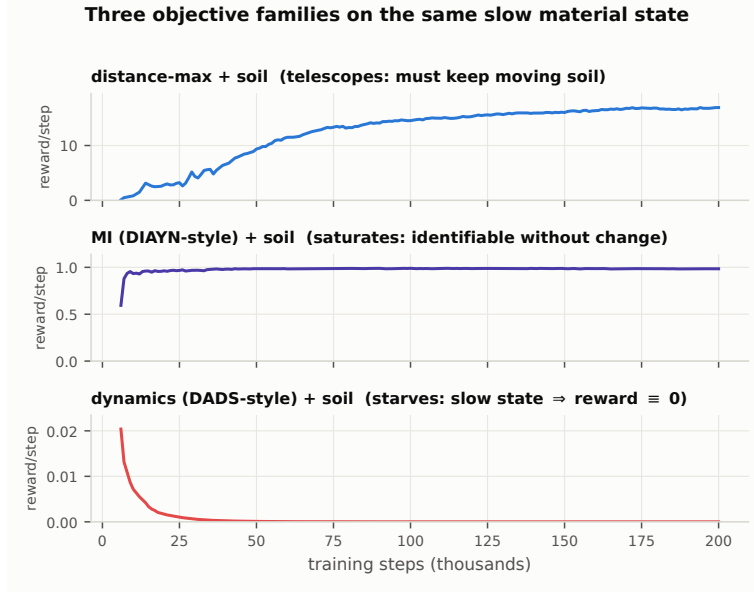


Figure 7: The three objective families’ intrinsic reward over training, all with soil-only conditioning, all on the same simulator. Top: distance-maximizing — reward *grows* as digging improves (it telescopes; only net transport pays). Middle: MI — *saturates* within a few thousand steps and flatlines. Bottom: dynamics-predictability — *starves* to exactly zero. This single figure is the taxonomy.

7.8 The unifying law: materials are slow states

Soil changes rarely and gradually; the arm changes richly at every step. Write each objective’s reward as a function of the state sequence and this one property sorts everything:

- **MI (DIAYN):** pays for per-step *identifiability*. Slow states still carry stable microscopic fingerprints $\Rightarrow$ maximal reward without macroscopic change $\Rightarrow$ **saturates**.
- **Dynamics (DADS):** pays for per-step *predictability advantage*. Slow states are trivially predictable under all skills $\Rightarrow$ the advantage is identically zero $\Rightarrow$ **starves**.
- **Distance-max (METRA family):** by the telescoping identity (2), pays for *net cumulative displacement* of the (soil) state $\Rightarrow$ the only reward that a slow state can still feed, because it *integrates* change over the episode instead of grading it per step $\Rightarrow$ **works**.

The two-ingredient design law

On slow external states (materials), a skill-discovery objective must **(i)** integrate change over time rather than score it per step (distance-maximizing / telescoping structure), and **(ii)** be conditioned *only* on the external state. In our 3×2 grid (three objective families $\times$ two conditionings, 2–3 seeds per cell), exactly the one cell with both properties works — at $8\times$ the coverage of the best cell lacking either.

7.9 Experiment 8 — Where it honestly breaks (and why)

Problem. An interactive demo: the user sculpts terrain with the mouse, records it as a goal, wrecks it, and the excavator should restore it. It performed poorly. Why?

Diagnosis chain (each hypothesis tested and dismissed in turn):

1. *Unfamiliar states?* We retrained *on* user-like edits (random push/remove/add operations at every reset). No improvement.
2. *Poor skill selection?* We built a model-predictive controller that uses **the simulator itself** as a perfect lookahead model: snapshot the world, mentally roll out every candidate skill, execute the provably best one. Improvement: +6–13% only.
3. With perfect selection eliminated, only one suspect remains: the **vocabulary**. The discovered skills span 3–5 coarse global transport modes; a freehand repair needs *local* corrections at arbitrary positions — directions of state-space the diversity objective never valued, because they contribute little diversity. Moreover user edits are typically *smaller than* $r(\varepsilon) \approx 0.2\text{ m}$: inside the granularity radius where the contraction certificate itself says progress stops.

Diversity $\neq$ controllability

Skill discovery optimizes for the few *most mutually different* behaviors, not for a dense basis of *all* behaviors. It yields coarse reachability of the state space — ideal as pretraining and for coarse commanded work; below its granularity radius, a task-optimized layer (e.g. goal-conditioned fine-tuning warm-started from the skills) is required. $r(\varepsilon)$ makes this boundary *measurable*. A supporting negative: a goal-conditioned SAC trained *from scratch* (no skill pretraining) on the same tasks fails to learn at all — 800k steps of contact-rich exploration finding no gradient — evidence that the skills’ role as *initialization* is real.

Two mechanical limits also surfaced, worth stating because they are physics, not learning: (a) one bucket interaction disturbs $\sim 0.1\text{ m}$ of W_2 — repairs smaller than a scoop are like surgery with a backhoe; (b) with a *fixed* base and backhoe geometry, soil transports only *toward* the machine — real operators solve this by driving around the pile, and giving our machine a mobile base is the corresponding fix.

7.10 The full frontier comparison

How to read the next three figures (a plain-language decoder)

The upcoming figures use four terms constantly. Here is each one, defined without jargon, with the concrete objects behind them.

μ (“mu”) — **the soil, written as a list of numbers.** Chop the 12.8 m soil bed into 64 vertical columns. Weigh the soil in each column. Divide by the total, so the 64 numbers sum to 1. That list *is* μ . Nothing more mysterious: $\mu = (0.011, 0.014, 0.017, \dots)$ means “1.1% of all the soil is in the leftmost column, 1.4% in the next, ...”. Two different soil landscapes give two different lists. A number like $W_2(\mu_a, \mu_b) = 0.3\text{ m}$ means: to physically rebuild landscape *a* into landscape *b*, a perfect bulldozer must move the soil, on average, 0.3 meters. When any figure says

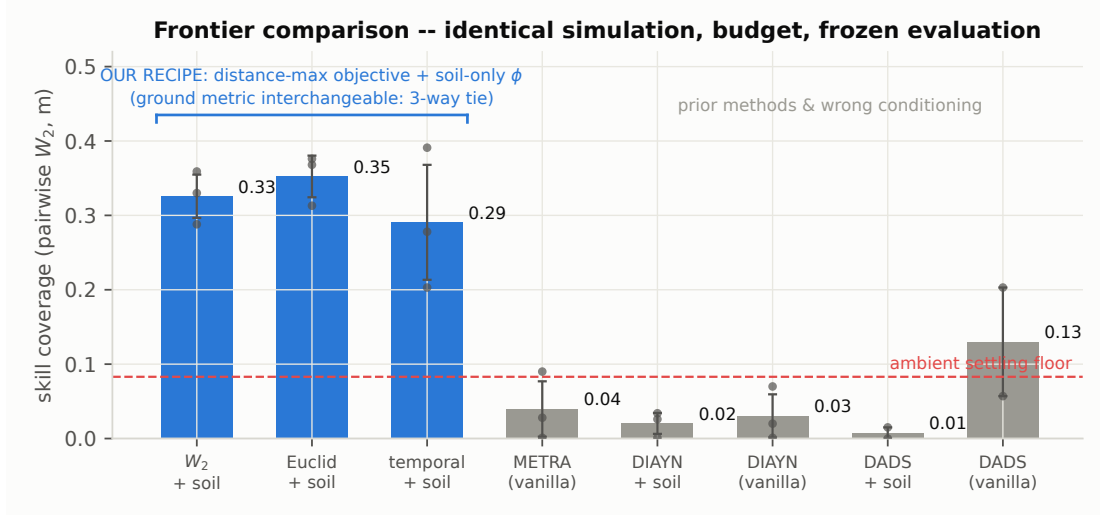


Figure 8: Everything on one axis: 8 method-variants, identical simulation, identical budget, frozen evaluation. Blue: distance-maximizing objectives with soil-only conditioning (any metric). Gray: every other family and conditioning. Dots are individual seeds; the dashed line is what soil does with a motionless arm.

method	corresponds to	coverage (m)	seeds
dist-max W_2 + soil ϕ	ours (TMSD)	0.326 ± 0.029	3
dist-max Euclidean + soil ϕ	ablation	0.351 ± 0.030	3
dist-max temporal + soil ϕ	METRA + our conditioning	0.290 ± 0.077	3
dist-max, full state	vanilla METRA	0.039 ± 0.036	3
MI + soil ϕ	DIAYN + our conditioning	0.021 ± 0.015	3
MI, full state	$\approx$ vanilla DIAYN	0.031 ± 0.028	3
dynamics + soil ϕ	DADS + our conditioning	0.008 ± 0.008	2
dynamics, full state	$\approx$ vanilla DADS	0.130 ± 0.073	2
sliced- W_2 , 2D measure	ours (negative result)	0.064	1

Table 1: The frozen-protocol comparison. Every vanilla frontier method fails on this domain; every distance-maximizing + soil-conditioned variant succeeds; metric choice within the working family is irrelevant. Configuration parity: same networks ($\sim 315k$ parameters), optimizer, budget (200k steps), terrain randomization, and evaluation; complete per-run data in `runs/full_comparison.csv`.

“coverage = 0.386 m”, the unit is meters *of required earth-moving* — a physical quantity, not an abstract score.

ϕ (“phi”) — **the learned summary of μ** . ϕ is a small neural network. You feed it the 64 numbers of μ ; it outputs just 4 numbers. Those 4 numbers are the state’s *coordinates on a learned map* — like reducing a whole country to a pin on a chart. The training constraint forces this map to be honest: two soil states end up far apart on the map *only if* turning one into the other requires a lot of real earth-moving. We call the map’s output space the **latent space** (“latent” = hidden/internal — coordinates the network invented, not ones we defined). When we plot “ ϕ_1 vs ϕ_2 ” we are simply plotting the first two of those 4 numbers.

Latent trajectory / latent spread. Run one episode: the soil changes step by step, so μ changes step by step, so the pin $\phi(\mu)$ *moves across the map*, tracing a curve — the **latent trajectory** of that episode. The **latent spread** of a method is simply how far its 8 skills’ curves fan out from the starting pin. Crucial warning, and the point of Figure 9: a big fan is meaningful

only if the map is honest. Vanilla METRA’s map is calibrated to time-steps, not to soil, so its skills draw a huge impressive fan while the actual soil never changes — a large map distance that corresponds to zero earth moved. Always check the latent picture against the physical one.

The coverage matrix — all skill pairs, one grid. Take the 8 skills. Run each from the identical starting terrain; photograph the final soil state of each ($\mu^{(1)}, \dots, \mu^{(8)}$). Now fill an 8×8 grid: the cell in row i , column j holds $W_2(\mu^{(i)}, \mu^{(j)})$ — the earth-moving distance between skill i ’s final world and skill j ’s final world. Bright cell = those two skills genuinely did different things to the soil. Dark cell = despite being “different skills,” they left the world in the same state. The diagonal is always black (every skill matches itself). The **coverage score** quoted everywhere in this report is just the average of all off-diagonal cells — “on average, how far apart are the worlds my skills produce?” Reading Figure 10 with this key: our quilt of bright cells means pairwise-distinct outcomes almost everywhere; METRA’s single bright cross means one skill did something and seven skills produced literally interchangeable worlds; DIAYN’s faint haze and DADS’s black square mean nobody moved anything.

Inside the latent spaces: why the bars differ

Numbers say *that* the methods differ; Figure 9 shows *why*, by opening each method’s own latent space next to what its skills physically accomplish — same 8 skills, same terrain for every row. The single most instructive quantity is the **scale of each latent map**:

method	latent spread (ϕ units)	coverage (m)	reading
METRA (vanilla)	~ 150	0.093	huge fan of <i>arm choreography</i>
ours	~ 2.5	0.386	modest fan of <i>soil transport</i>
DIAYN + soil	~ 0.002	0.036	saturation jitter
DADS + soil	~ 0.00007	0.002	starvation flatline

Vanilla METRA’s row is the arm-dancing indictment in one image: a textbook eight-direction latent fan — $60\times$ wider than ours — above terminal soil profiles that lie *exactly on the initial terrain*. Its temporal constraint grants ~ 1 latent unit per time step *regardless of what changed*, so 200 steps of choreography buy a ± 150 fan; latent diversity is decoupled from the world. Ours is the only row where the two columns agree: the W_2 cap makes every latent unit cost real mass transport, so a modest fan *means* five distinct excavations. Below, DIAYN’s saturation and DADS’s starvation are visible as sheer magnitude: their latent spaces barely move at all. The taxonomy of Section 7.8, readable as four axis scales.

Finally, Figure 10 shows the raw object behind every “coverage” number in this report: the 8×8 matrix of pairwise W_2 distances between the skills’ terminal soil states — on **one shared color scale**, which per-method auto-scaled heatmaps would destroy. Read it like a similarity fingerprint: bright cell = that pair of skills left the world in genuinely different states. Ours is a quilt — distinctness almost everywhere. Vanilla METRA is a single bright cross: exactly one skill (#7) ever moved soil, so only its row and column light up, while the other seven skills — maximally diverse *in latent space* — are pairwise identical *in the world*. DIAYN is a faint ghost of structure at saturation level; DADS is uniformly black.

8 What is actually new here (novelty statement)

1. **First demonstration of reward-free skill discovery on granular media.** Digging, piling, and directed transport emerge with no rewards; prior USD work covers locomotion and rigid objects only.
2. **The two-ingredient law with a $3 \times 2 \times \text{seeds}$ factorial.** Cumulative (distance-maximizing) objectives *and* external-only conditioning are each necessary, jointly sufficient; $8\times$ effect size.

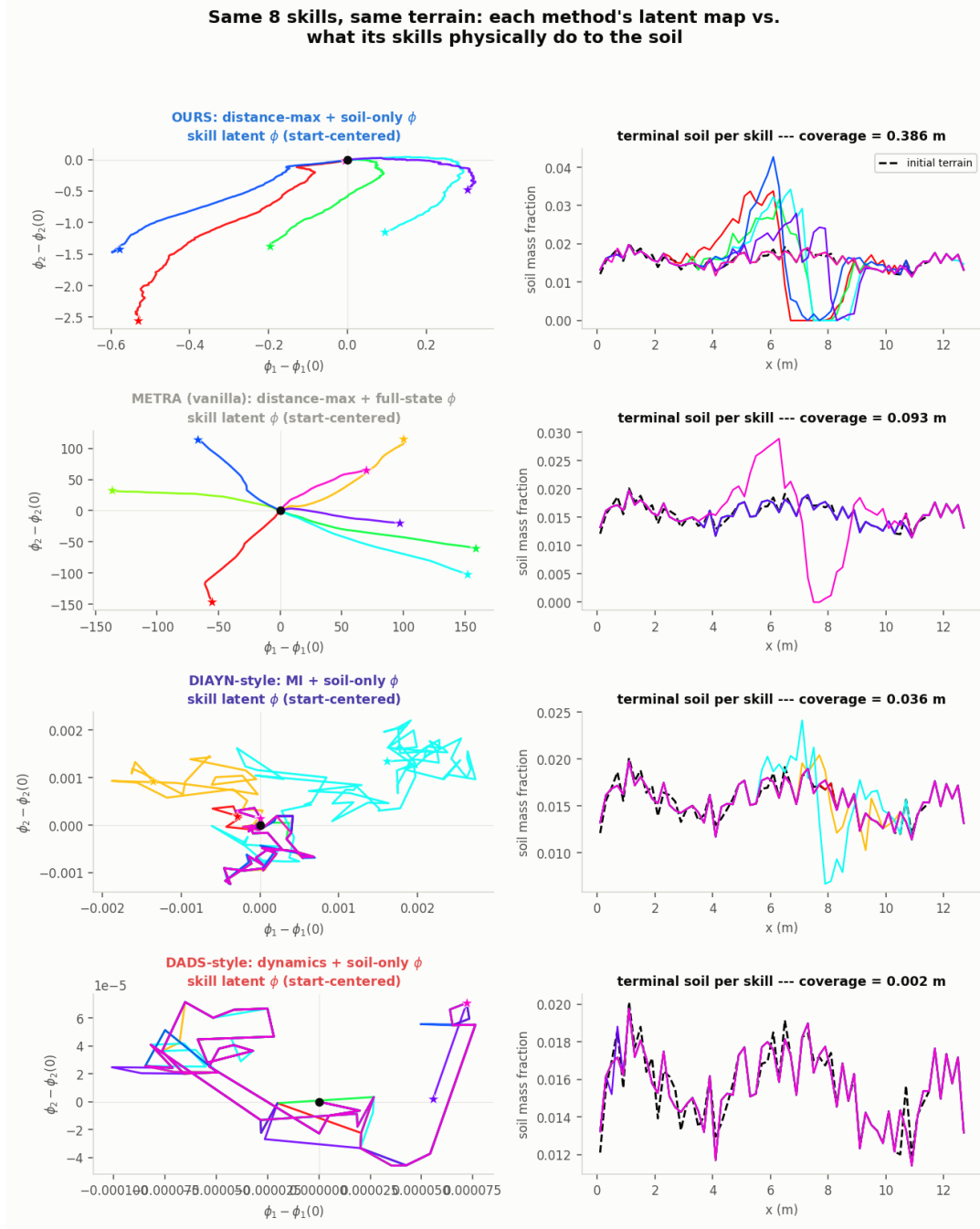


Figure 9: Same 8 protocol skills, same terrain, one representative checkpoint per family. **Left column:** each model's own latent trajectories (start-centered; star = terminal; note the wildly different axis scales). **Right column:** the terminal soil profile of each skill vs. the initial terrain (dashed), with the physical coverage in the title. Latent spread and physical accomplishment agree *only* for our recipe.

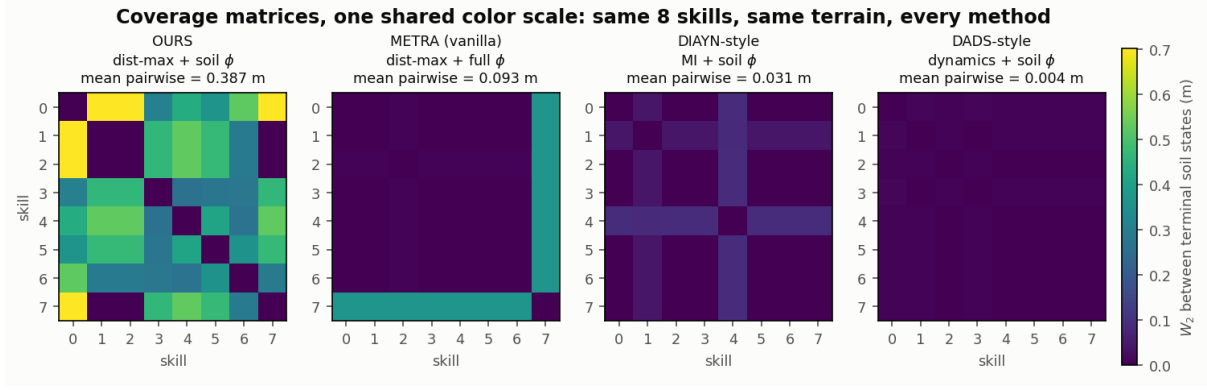


Figure 10: Pairwise- W_2 coverage matrices, same 8 skills, same terrain, shared color scale. The mean of each matrix’s off-diagonal is that method’s coverage score in the frontier table.

To our knowledge the first controlled factorial of objective family $\times$ discriminator conditioning anywhere in USD.

- Metric-agnosticism, measured and explained.** Within the working cell the ground metric is irrelevant (three-way tie), because every ϕ becomes a 0.99-isometry of physical transport regardless — an emergent-geometry result that runs *counter* to the METRA lineage’s emphasis (and to our own original hypothesis, which we falsified with pre-registered ablations).
- The slow-state failure taxonomy** — saturation (MI) / starvation (dynamics) / telescoping (distance-max) — turning three unrelated-looking failures into one design law for material manipulation objectives.
- Certified zero-shot shaping.** Latent steering achieves 39–63% goal-distance reduction with $\sim 90\%$ monotone contraction and a measurable granularity radius $r(\varepsilon)$ — the quantitative boundary between what skill composition can and cannot reach (= “diversity $\neq$ controllability,” with a number).

9 Limitations and what comes next

Limitations, plainly. One domain family in one custom simulator; the 1D measure dominates (vertical structure underexplored — and the one 2D-metric run failed by starvation, unexplained at the optimization level); skill granularity $\approx 0.2\text{m}$ bounds precision; fixed base bounds reachability; sample sizes are 2–3 seeds per cell (effect sizes are large relative to seed noise, but more seeds are cheap insurance); the MI cell is DIAYN-style skill-MI, not MUSIC’s agent-surroundings MI (a faithful MUSIC variant remains to be run).

Next steps, in order of importance.

- Second material* (2D dough/plasticine): the objective ports unchanged; success turns the two-ingredient law from a finding about soil into a principle about materials.
- Mobile base + goal-conditioned layer warm-started from skills*: removes the two mechanical/precision limits; the from-scratch control experiment already shows the warm start is the necessary half.
- Contraction theorem*: formalize “ ε -coverage of transport directions $\Rightarrow$ geometric W_2 contraction to radius $r(\varepsilon)$ ” — the 90%-monotone data is its empirical half.
- Write-up*: empirical paper (CoRL/RSS shape) around items 1–3.

A Appendix: reproduction guide

Hyperparameters (all runs). SAC: lr $3 \cdot 10^{-4}$, batch 256, γ 0.99, τ 0.005, auto-tuned α (target entropy -3), twin critics, buffer $3 \cdot 10^5$. Networks: 2-hidden-layer MLPs, width 256 (~ 315 k parameters total per model). TMSD: skill dim 4 (pilot: 2), dual lr 10^{-3} , $\lambda_0 = 30$, relative slack 0.25, reward scale 100 (W_2 rewards are $\sim 10^{-3}$ m per step and would drown in SAC's entropy term otherwise), W_2 quadrature 2048 quantiles. DADS: $\sigma = 0.01$, $L = 16$. Episodes 200 steps; terrain randomization: random cosine height envelope scaled to bed height.

Key commands.

- Train: `python scripts/train.tmsd.py --steps 200000 --skill-dim 4 --randomize-terrain [--metric w2|euclidean|temporal|sliced.w2] [--objective metra|mi|dads] [--phi-input hist|obs] [--measure 1d|2d]`
- Evaluate (frozen protocol): `python scripts/compare_runs.py --all-batch`
- Goal-reaching study: `python scripts/eval_goal_reaching.py`
- Demos: `python scripts/showcase.py` (3-act public demo), `python scripts/demo_excavate_dump.py --dig 7 9 --dump 4.5 6.5 --live`, `python scripts/interactive_demo.py`
- All figures in this report: `python report/make_figures.py`

Data inventory. 33 runs under `runs/`: `per-run log.csv` (training telemetry), `episodes.csv` (per-episode soil transport), checkpoints; `runs/compare_summary.csv` (frozen-protocol scores, both yardsticks); `runs/full_comparison.csv` (configs, parameter counts, wall-clock, throughput, eval merge); `runs/goal_eval/` (steering and isometry data). Narrative documents: `Ideation.md` (the original plan and its pre-registered pivot rules), `Findings.md` (the verdict trail), `Comparison.md` (the final table).